



React JS - Day 2 Notes

Course Progress: ☒ React Day 2 Completed



Topics Covered

- GSAP Overview
- React Introduction
- React DOM
- Creating First React Project Using CDN
- `React.createElement()`
- `ReactDOM.createRoot()`
- `render()`
- Rendering Multiple Elements
- Props
- Inline Styling
- `className`



GSAP Overview

What is GSAP?

GSAP (GreenSock Animation Platform) is a powerful JavaScript animation library used for creating fast, smooth, and professional animations.

Import GSAP CDN

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/gsap/3.13.0/gsap.min.js"></script>
<script type="module" src="script.js"></script>
```

Basic Animation

```
gsap.to("#box", {  
  x: 500,  
  duration: 1,  
  delay: 1,  
});
```

Explanation

Property	Meaning
x	Moves the element horizontally
duration	Animation duration in seconds
delay	Waits before starting animation



React Overview

React JS

React is a JavaScript library used to build **User Interfaces (UI)**.

React JS → Used for Building UI

React DOM

ReactDOM connects the **Virtual DOM** with the **Real DOM** (Browser DOM).

It efficiently updates only the changed parts of the page.

ReactDOM → Bridge Between React and Browser



Creating First React Project Using CDN

Include React and ReactDOM CDN files **before** your JavaScript file.

To verify whether React is loaded successfully:

```
console.log(React);
```

```
console.log(ReactDOM);
```

If both objects are printed in the console, React is connected successfully.

React.createElement()

Syntax

```
React.createElement(elementName, props, children)
```

Example

```
React.createElement("h1", null, "Hello From React");
```

Parameters

Parameter	Description	Example
<code>elementName</code>	HTML tag name to create	<code>"h1"</code> , <code>"div"</code> , <code>"button"</code>
<code>props</code>	Attributes or properties of the element	<code>{ id: "title" }</code> , <code>{ className: "box" }</code> , <code>{ style: { color: "red" } }</code>
<code>children</code>	Text content or nested elements	<code>"Hello"</code> , <code>h1</code> , <code>[h1, h2]</code>

Selecting Root Element

```
let selectedRoot = document.querySelector("#root");
```

Note: Never use `<body>` as the React root element. Always create a separate `<div id="root"></div>` and render your React application inside it.

Creating React Root

```
let root = ReactDOM.createRoot(selectedRoot);
```

This creates a React Root that controls everything inside the selected element.



Rendering an Element

```
root.render(h1);
```

The `render()` method displays the React element inside the root container.



Complete Example

```
let h1 = React.createElement(  
  "h2",  
  null,  
  "Hello From React"  
);  
  
let selectedRoot = document.querySelector("#root");  
  
let root = ReactDOM.createRoot(selectedRoot);  
  
root.render(h1);
```



Incorrect Way to Render Multiple Elements

```
root.render(h1, h2);
```

 Wrong

The `render()` method accepts only **one root element**.



Solution 1: Wrap Elements Inside a Parent

```
let h1 = React.createElement(  
  "h1",  
  null,  
  "Hello From React 1"  
);  
  
let h2 = React.createElement(  
  "h2",  
  null,  
  "Hello From React 2"  
);  
  
let div = React.createElement(  
  "div",  
  null,  
  [h1, h2]  
);  
  
let selectedRoot = document.querySelector("#root");  
  
let root = ReactDOM.createRoot(selectedRoot);  
  
root.render(div);
```



Solution 2: Render as an Array

```
root.render([h1, h2]);
```

React may show a warning because every element inside an array should have a unique **key**.



Props

The second argument of `React.createElement()` is called **props**.

Props are used to pass attributes to an element.

Example

```
React.createElement(  
  "h1",  
  {  
    id: "title",
```

```
    },
    "Hello"
  );
```



Inline Styling

The `style` property must always be written as an **object**.

```
React.createElement(
  "h1",
  {
    style: {
      color: "red",
      backgroundColor: "black",
      padding: "10px",
    },
  },
  "Hello From React"
);
```



Using className

React uses `className` instead of HTML's `class`.

```
let div = React.createElement(
  "div",
  {
    className: "container",
  },
  [h1, h2]
);
```

✅ Correct

```
className: "container"
```

❌ Wrong

```
class: "container"
```



Important Notes

- React is used to build user interfaces.
 - ReactDOM connects React with the browser.
 - `React.createElement()` creates Virtual DOM elements.
 - `ReactDOM.createRoot()` creates a React Root.
 - `render()` accepts only one root element.
 - Multiple elements should be wrapped inside a parent element.
 - Props are passed as the second argument.
 - `style` must always be an object.
 - Use `className` instead of `class`.
 - Arrays rendered in React should have unique keys.
-